

Introduction to Parallel Algorithms and Programming

Session 3: Multicore programming with OpenMP

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- Run a PRAM-style program on a **multicore** processor: PU i becomes **thread i**
- Learn a **minimal toolbox** of OpenMP, one construct at a time, each on a running example:
 - create and destroy threads: `parallel`, `num_threads`
 - know who you are: `omp_get_thread_num`, `omp_get_num_threads`
 - what threads share, what they own: the two rules for variables
 - synchronize asynchronous threads: `barrier`
 - distribute work by thread id, then by sections
- Understand the one thing the PRAM did for us that the machine does not:
concurrent writes
- Measure the time, compute the speedup; meet the **race condition** and the way out
- Not today: the rest of OpenMP (automatic loop distribution, tasks, ...): the HPC on multicore course

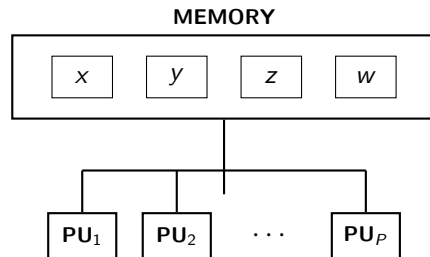
Outline

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

What we learned in session 2: the PRAM

- P processing units (PUs) and one memory, all connected
- Every PU runs the **same program** on its **own index** i ; it also knows P (or N)
- Data lives in the memory; each PU has **private registers**: load, operate on registers, store
- Parallel work is written **forall** $i = 1, \dots, N$ **in parallel do** ...
- **Synchronous**: all PUs execute the same line at the same step; in a step, all reads happen before all writes
- Memory conflicts are settled by the model: EREW, CREW, CRCW (with a concurrency mode)



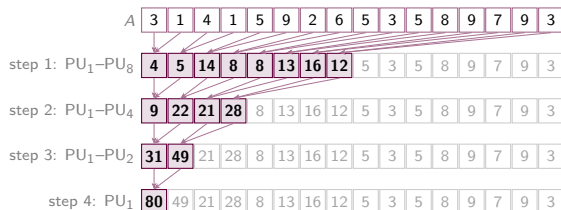
One PRAM algorithm: sum of an array

SUMARRAYEREW($A[1 \dots N]$)

```

1:  $n \leftarrow N$ 
2: while  $n > 1$  do
3:   forall  $i = 1, \dots, \lceil n/2 \rceil$  in parallel
     do
4:     if  $2i \leq n$  then
5:        $A[i] \leftarrow A[2i - 1] + A[2i]$ 
6:     else
7:        $A[i] \leftarrow A[2i - 1]$ 
8:    $n \leftarrow \lceil n/2 \rceil$ 
9: return  $A[1]$ 
    
```

- In place: at each step, PU i reads $A[2i - 1]$, $A[2i]$ and writes **its own cell** $A[i]$; the first half of A is reused, the rest is dead
- PU i is active only while $i \leq n/2$; the others fail the test and **idle**. $N = 16$: 8, 4, 2, 1 PUs active, $\lceil \log_2 N \rceil = 4$ steps, $O(N)$ work
- PU i writes $A[i]$ while PU $\lceil i/2 \rceil$ reads it: fine, **reads happen before writes** in a step; the next step starts when everyone has written



université
PARIS-SACLAY

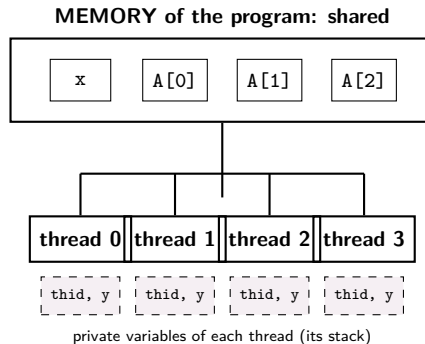
PRAM	Multicore with threads	Today
PU i , one per data element (N PUs)	thread i , one per core: a handful of threads, far fewer than data elements	<code>parallel num_threads(P), omp_get_thread_num()</code>
memory; private registers	variables of the sequential part : shared; variables of the parallel block : private	the two rules for variables
lockstep: one line per step, reads before writes	threads run at their own pace ; nothing is synchronized unless we ask	<code>barrier</code> (and the implicit ones)
conflicts settled by the model (EREW, CREW, CRCW)	CREW-like : reading the same variable at once is fine, writing it at once is a bug, and our problem	the rule: write your own cell, combine after

Same idea (one program, one index per worker), four differences. The session fills this table in.

- 1 From the PRAM to threads
- 2 Threads and OpenMP**
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

What is a thread?

- A running program is a **process**: its memory (code, global variables, heap) and one **execution stream** (where we are in the program, plus a stack for local variables)
- A **thread** is one execution stream: its own position in the program and its own stack, in the **same memory** as the other threads of the process
- The operating system runs each thread on a core: P threads on P cores work **simultaneously**
- More threads than cores: the cores take turns; it still works, but no faster



What is OpenMP?

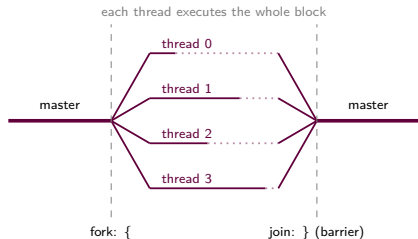
- An extension of C, C++ and Fortran for programming with threads on shared memory (since 1997), supported by gcc, clang, MSVC, ...
- Three ingredients:
 - **directives**: `#pragma omp ...` placed just before a block `{ ... }`; they tell the compiler what to do with that block
 - **runtime functions**: `omp_get_thread_num()`, `omp_get_num_threads()`, ..., declared in `<omp.h>`
 - environment variables (`OMP_NUM_THREADS`, ...): change the behaviour without recompiling; we will not need them
- Compile with **`-fopenmp`**. Without it, the directives are **ignored**: the program compiles and runs, sequentially, with the same result
 - a feature: one source, sequential or parallel; and a trap: no speedup and no error message
- OpenMP is large. Today: **four directives and two functions**, which are enough to write every PRAM algorithm of session 2 as a program

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

Creating threads: parallel

```
#pragma omp parallel num_threads(P)
{
    // executed by each of the P threads
}
// only the master thread continues
```

- **Fork:** P threads are created (thread 0 is the master, the one that arrived); each executes the **whole block**
- The threads execute the block **asynchronously**: at a given moment they can be at different lines of it
- **Join:** at the closing brace, every thread waits for the others (**implicit barrier**); then threads $1 \dots P - 1$ end, the master goes on alone
- P : a constant or a variable; without `num_threads`, OpenMP picks a default (usually the number of cores)



Example: Oh, no no no! (01-hello.cpp)

```
1 #include <stdio>
2
3 int main()
4 {
5     printf("Oh,");
6
7     #pragma omp parallel num_threads(3)
8     {
9         printf(" no");
10    }
11
12    printf("!\n");
13    return 0;
14 }
```

- Output: Oh, no no no!
- Line 9 is executed **three times**, once by each thread; nothing in the block tells the three threads apart, so nothing tells the three nos apart either
- Line 12 runs after the join: the master alone prints !, once, after the last no
- Compiled without `-fopenmp`: Oh, no!
- Creating and joining threads has a cost (tens of microseconds): not for tiny blocks

Who am I? Thread id and number of threads

```
#include <omp.h>
...
#pragma omp parallel num_threads(P)
{
    int thid = omp_get_thread_num();
    int numth = omp_get_num_threads();
    ...
}
```

- `omp_get_thread_num()`: my id, between 0 and $P - 1$: the **index** i of the PRAM (but numbered from 0)
- `omp_get_num_threads()`: the number of threads of the region: the P of the PRAM
- Outside a parallel region they return 0 and 1: the master is a thread, alone
- Store them in variables declared **inside** the block (next slide), by convention `thid` and `numth`
- `thid` is how we distribute work among threads: exactly the PRAM way of programming

Example: hello with ids (02-hello-ids.cpp)

```

1  #include <stdio>
2  #include <omp.h>
3
4  int main()
5  {
6      printf("before: thread %d of %d\n",
7             omp_get_thread_num(), omp_get_num_threads());
8
9      #pragma omp parallel num_threads(4)
10     {
11         int thid = omp_get_thread_num(); // my index
12         int numth = omp_get_num_threads(); // how many of us
13         printf("I am thread %d of %d\n", thid, numth);
14     }
15
16     printf("after: thread %d of %d\n",
17            omp_get_thread_num(), omp_get_num_threads());
18     return 0;
19 }

```

- Output: before: thread 0 of 1,
four lines I am thread ...of 4,
after: thread 0 of 1
- Run it again: the four lines come in a **different order**. Threads are asynchronous; nothing says who prints first
- Lines 11–12: each thread has **its own** thid and numth: they are declared inside the block
- On a real machine, every thread of this program will do the same job on a different index; ids are the only difference

université
PARIS-SACLAY

```

1  #include <stdio>
2  #include <omp.h>
3
4  int main()
5  {
6      int x = 3;                // before the block: shared
7      int A[4] = {0, 0, 0, 0};  // before the block: shared
8
9      #pragma omp parallel num_threads(4)
10     {
11         int thid = omp_get_thread_num(); // inside: private
12         int y = x + thid;                // inside: private
13         A[thid] = y;                     // my own cell of A
14         printf("thread %d: x = %d, y = %d\n", thid, x, y);
15     }
16
17     printf("A = %d %d %d %d\n", A[0], A[1], A[2], A[3]);
18     return 0;
19 }

```

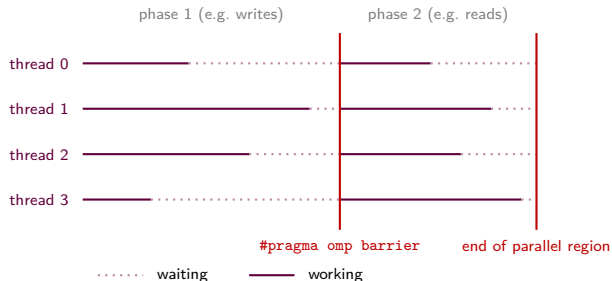
- Output: four lines thread t : $x = 3$, $y = 3+t$ (in some order), then $A = 3 \ 4 \ 5 \ 6$
- x and A : one copy, read (x) or written (A) by all threads
- $thid$, y : four copies, one per thread; the value of y differs on each
- Line 13: thread $thid$ writes **its own cell** $A[thid]$, nobody else touches it: the EREW discipline, and how we will always write results
- At the end of the block the four y are **destroyed**; they survive only because line 13 stored them in A (sequential part). After the join, the master reads A

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers**
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

Threads are asynchronous; the barrier

- **No lockstep**: each thread advances at its own pace, and the operating system may pause any thread at any moment
- Two guarantees only: at the end of the parallel region, every thread has finished the block; and at a **barrier**, no thread goes on before all P threads have arrived
- A PRAM step is “everyone reads, then everyone writes”. Between two phases of a program that must not overlap: a barrier
- **Every thread** of the region must reach the barrier: never inside if (`thid == ...`), or the others wait forever

```
#pragma omp parallel num_threads(P)
{
    ...           // phase 1
    #pragma omp barrier
    ...           // phase 2: sees all of phase 1
}
```



Example: broadcast (04-bcast.cpp)

```

1  #include <stdio>
2  #include <unistd.h>
3  #include <omp.h>
4
5  int main()
6  {
7      int e = 0;
8
9      #pragma omp parallel num_threads(4)
10     {
11         int thid = omp_get_thread_num();
12         if (thid == 0) {
13             usleep(100000);    // thread 0 takes 0.1 s to produce e
14             e = 42;
15         }
16         #pragma omp barrier    // wait here until everyone is here
17         printf("thread %d sees e = %d\n", thid, e);
18     }
19     return 0;
20 }
```

- Thread 0 produces a value, the others need it: the BCAST problem of session 2
- Output: thread t sees $e = 42$ for all four threads
- Remove line 16: threads 1–3 arrive at line 17 while thread 0 is still asleep and print $e = 0$; the output depends on timing
- On an EREW PRAM, BCAST took $\lceil \log_2 N \rceil$ steps of copying. With shared memory: one write and one barrier. The barrier is where the cost went: every thread stops, and its price grows with P

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id**
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

The problem: computing nine (05-nine-seq.cpp)

```

6 double now() { // wall-clock time in seconds (not OpenMP)
7     using namespace std::chrono;
8     auto t = high_resolution_clock::now().time_since_epoch();
9     return duration<double>(t).count();
10 }
11
12 #define UNIT 100000 // 1 unit of time = 0.1 s (microseconds)
13 int computeTwo() { usleep(2 * UNIT); return 2; }
14 int computeThree() { usleep(3 * UNIT); return 3; }
15 int computeFour() { usleep(4 * UNIT); return 4; }
16
17 int main()
18 {
19     double t0 = now();
20
21     int two = computeTwo();
22     int three = computeThree();
23     int four = computeFour();
24     int nine = two + three + four;
25
26     double t1 = now();
27     printf("nine = %d, computed in %.2f s\n", nine, t1 - t0);
28     return 0;
29 }

```

- Three **independent** tasks: computeTwo, computeThree, computeFour take 2, 3 and 4 units of time (one unit is 0.1 s of sleep) and return 2, 3, 4
- Sequentially: $2 + 3 + 4 = 9$ units, nine = 9, computed in 0.90 s
- With three threads, one task each: the parallel time is the **longest task**, 4 units, not $9/3$: speedup 2.25, efficiency 75%
- Timing: the helper now() (lines 6–10) reads the wall clock in seconds with C++ std::chrono, as in lab 1; not OpenMP

Computing nine: a first attempt (06-nine-redundant.cpp)

```

17 int main()
18 {
19     int two = 0, three = 0, four = 0, nine = 0;
20     double t0 = now();
21
22     #pragma omp parallel num_threads(3)
23     {
24         two = computeTwo();
25         three = computeThree();
26         four = computeFour();
27         nine = two + three + four;
28     }
29
30     double t1 = now();
31     printf("nine = %d, computed in %.2f s\n", nine, t1 - t0);
32     return 0;
33 }

```

- Three threads, and the three calls in the parallel block: **what is wrong with this program?**
- Output: nine = 9, computed in 0.90 s: correct, and **no faster** than the sequential version
- Every thread executes the **whole block**: each function is called three times, once per thread. The work is done **redundantly**, not **shared**
- The three threads also write the same shared two, three, four, nine: same values, so the result is right, by luck
- To share the work, each thread must do something **different**: only their ids tell them apart

Computing nine by thread id (07-nine-thid.cpp)

```

17 int main()
18 {
19     int two = 0, three = 0, four = 0;
20     double t0 = now();
21
22     #pragma omp parallel num_threads(3)
23     {
24         int thid = omp_get_thread_num();
25         if (thid == 0) two = computeTwo();
26         if (thid == 1) three = computeThree();
27         if (thid == 2) four = computeFour();
28     } // implicit barrier: two, three, four are ready
29
30     int nine = two + three + four;
31
32     double t1 = now();
33     printf("nine = %d, computed in %.2f s\n", nine, t1 - t0);
34     return 0;
35 }

```

- Lines 25–27: work **dispatched by id**: thread 0 computes two, thread 1 three, thread 2 four; each thread skips the two lines that are not for it
- Each thread writes a **different** shared variable: no conflict
- Line 30: after the join, the three values are ready; the master adds them
- Output: nine = 9, computed in 0.40 s: the longest task
- The PRAM style: “PU i does the work of index i ”, with `if (thid == i)` instead of **forall**

The limits of dispatching by id

Same program, 07-nine-thid.cpp, and we change only the number of threads on line 22.

- `num_threads(2)`: thread 2 does not exist, line 27 is never executed, four stays 0: **nine = 5**, in 0.30 s, with **no error and no warning**. The program is wrong only because of P
- `num_threads(4)`: thread 3 has nothing to do and idles until the join: correct, but a core is wasted
- Ten tasks and three threads: we must decide ourselves which task goes to which id, write it as ifs, and redo it when P changes
- What we really mean: here are three **independent tasks**, to run in parallel. We do not care whether thread 0 or 2 computes four; we want an **available** thread to take it as soon as possible
- So: describe the tasks, and let OpenMP hand them to whatever threads exist: **sections**

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`**
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

Work sharing: sections

```
#pragma omp parallel num_threads(P)
{
  #pragma omp sections
  {
    #pragma omp section
    { f(); } // some thread does this
    #pragma omp section
    { g(); } // some thread does this
  } // implicit barrier
}
```

- Inside a parallel region, a sections block lists independent pieces of work; each section is executed **exactly once**, by **some** thread: which one is not ours to know
- The P threads take sections until none is left: more sections than threads, some threads do several; fewer, some threads idle
- **Implicit barrier** at the end of the sections block: after it, all sections are done
- Tasks are **decoupled** from ids: the same code is correct for any P
- The one-section trick: a sections block with a single section means “**one thread does this**” (and everybody waits): sequential work inside a region

Computing nine with sections (08-nine-sections.cpp)

```

22 #pragma omp parallel num_threads(3)
23 {
24     int thid = omp_get_thread_num();
25     #pragma omp sections
26     {
27         #pragma omp section
28         { two = computeTwo();
29           printf("two: %d\n", thid); }
30         #pragma omp section
31         { three = computeThree();
32           printf("three: %d\n", thid); }
33         #pragma omp section
34         { four = computeFour();
35           printf("four: %d\n", thid); }
36     } // implicit barrier: two, three, four are ready
37
38     #pragma omp sections // a single section: one thread does it
39     {
40         #pragma omp section
41         { nine = two + three + four;
42           printf("sum: %d\n", thid); }
43     }

```

- Lines 25–36: one section per task; each prints the thread that took it, e.g. two: 1, three: 2, four: 0
- Lines 38–43: one section, so one thread adds, **after** the implicit barrier of line 36: the three values are ready
- Output: nine = 9, computed in 0.40 s
- Change num_threads(3) to 2, 4 or 8: still **correct**, nothing else to change
- With 2 threads: 0.60 s: sections are handed out **in order** to whoever is free, so four starts last. **Load balancing** is now OpenMP's problem, solved greedily: four alone on a thread would give 0.50 s

Two ways to distribute work

By thread id (the PRAM way)

- if (`thid == ...`): the thread decides from its id what it has to do
- For **regular** work: N cells, thread `thid` takes its own (`A[thid]`)
- Every PRAM algorithm of session 2 is written this way
- We control everything: which thread touches which cell, hence where the conflicts are

By sections

- The program lists the tasks; OpenMP assigns them to threads
- For a **few different** tasks: three functions, two halves of a computation, one sequential step inside a region
- Correct for any number of threads
- We do not control who does what: fine as long as the tasks are independent and write different variables

In both cases, one question decides whether the program is correct: **do two threads ever write the same variable at the same time?**

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes**
- 8 Compiling, running, and the lab assignment

A shared variable written by several threads (09-race.cpp)

```

1  #include <stdio>
2  #include <omp.h>
3  #define N 1000000
4
5  int main()
6  {
7      int n = 0;    // 4 threads x N increments: counts to 4N
8
9      #pragma omp parallel num_threads(4)
10     {
11         for (int k = 0; k < N; k++)
12             n = n + 1;           // all 4 threads write n
13     }
14
15     printf("n = %d, expected %d\n", n, 4 * N);
16     return 0;
17 }
```

- Four threads count together to $4N = 4\,000\,000$: each adds 1 to the shared n , N times (lines 11–12)
- Output: $n = 1172705$, a **different** wrong value at every run
- Line 12 is three instructions: **load** n , **add** 1, **store** n . Two threads load the same old value, both store old +1: one increment is **lost**
- CRCW fusion mode would have added everything; EREW would have forbidden it. A real machine does neither: silently wrong. A **race condition**
- Compiled with `-O0` so that line 12 runs as written (with `-O2`: one store per thread, rarer and bigger losses)

The rule, and the way out (10-race-fixed.cpp)

```
1 #include <stdio>
2 #include <omp.h>
3 #define N 1000000
4
5 int main()
6 {
7     int n = 0, partial[4]; // partial[t]: count of thread t
8
9     #pragma omp parallel num_threads(4)
10    {
11        int thid = omp_get_thread_num();
12        int mine = 0;          // private accumulator
13        for (int k = 0; k < N; k++)
14            mine = mine + 1;
15        partial[thid] = mine;  // my own cell, no conflict
16    } // implicit barrier: all the cells are written
17
18    for (int t = 0; t < 4; t++) // one thread combines
19        n = n + partial[t];
20
21    printf("n = %d, expected %d\n", n, 4 * N);
22    return 0;
23 }
```

- **The rule:** never write the same shared variable from two threads at once
- **The way out**, the EREW discipline: each thread counts in a **private** accumulator (lines 12–14), stores it in **its own cell** (line 15), and one thread **combines** the partial sums after the join (lines 18–19)
- Output: $n = 4000000$, every time; cost: P additions, negligible when $N \gg P$
- Private accumulate / own cell / combine works for any operation (sum, max, count): the only pattern of this course

- 1 From the PRAM to threads
- 2 Threads and OpenMP
- 3 Creating threads: `parallel`
- 4 Asynchronous threads and barriers
- 5 Distributing work by thread id
- 6 Work sharing with `sections`
- 7 The one rule: concurrent writes
- 8 Compiling, running, and the lab assignment

Compiling and running

Compiler Explorer

C++, x86-64 gcc, options `-O2 -fopenmp`, “Execute the code” ticked in the “Output...” menu; the examples are single files: paste and run.

On your laptop (needed to measure speedups: Compiler Explorer may give you a single core)

Linux, WSL: `g++ -O2 -fopenmp prog.cpp -o prog && ./prog`

macOS: `brew install gcc`, then `g++-16` (g++ is secretly clang++: no OpenMP)

Windows: WSL to install Ubuntu, then g++ as on Linux

Summary: the dictionary, filled in

PRAM	OpenMP
P PUs	<code>#pragma omp parallel num_threads(P)</code>
index i , P	<code>omp_get_thread_num(), omp_get_num_threads()</code>
memory	variables declared before the block
private registers	variables declared inside the block
lockstep, reads before writes	<code>#pragma omp barrier</code> between phases
EREW discipline	write your own cell, combine after
CRCW fusion write	does not exist: a race condition

- The toolbox, six items: `parallel`, `num_threads`, `barrier`, `sections`, `section`; `omp_get_thread_num`, `omp_get_num_threads`
- Variables: declared before the block, shared; inside, private
- Correctness: never two threads writing the same variable at once
- Asynchronous threads: a barrier between phases that must not overlap; a PRAM step is read phase, barrier, write phase, barrier

Lab 2: multithreaded programming with OpenMP

Compiler Explorer (<https://godbolt.org>)

C++, x86-64 gcc, options `-O2 -fopenmp`, “Execute the code” ticked in the “Output...” menu; timings on your laptop with the `now()` timer of the skeletons. Only the constructs of today.

Ex. 1: hello (hello.cpp)

- 4 threads, each prints its id and the number of threads
- Then the master says hello; then one thread chosen by OpenMP: in this order, always
- Run again, with 2 and 8 threads: what changes?

Ex. 2: sum (sum.cpp)

- $N = 10^7$, $A[i] = i$; sequential init and sum
- Init with 4 sections, sum with 4 sections: local accumulator, own cell, sequential merge
- 1, 2, 4, 8 threads: speedup, efficiency; why no faster beyond 4?

Ex. 3: mergesort (mergesort.cpp)

- 4 sections sort the 4 quarters (mergesort given)
- 2 sections merge quarters into halves (merge given), then one merge
- Speedup with 4 threads: why below 4? 8 cores: 8 threads

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Paris, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com